

Lab Session 3

Nested Vectored Interrupt Controller

Table of Contents

1. Objectives of the Laboratory Experiment.....	1
2. Preparation.....	1
1 Laboratory Tasks.....	2
1.1 Interrupt based Distance Measurements.....	2
1.2 Interrupt based Pendulum Display.....	2
Appendix 1: How do I configure my own Interrupt-Service-Routine?.....	3

1. Objectives of the Laboratory Experiment

In this experiment, you will become familiar with the nested vectored interrupt controller of the processor. Your task is to implement the distance measurement from the previous lab without using any busy-waiting and polling.

2. Preparation

The Lab session will begin with a written test. You are expected to know the functionalities of the General Purpose Timer registers. You are expected to know how the prescaler works and is calculated, as well as which flags to poll for timeout and match events and how to busy-wait while polling the flags for TimerA and TimerB.

To prepare for the lab task, please do the following steps:

- ☐ I have read the experiment instructions thoroughly and have an idea of how to implement them.
- ☐ I have familiarized myself with the lecture content on interrupts.
- ☐ I have thought about how to configure the GPIO port registers and the general-purpose timer for the tasks with interrupts.

1 Laboratory Tasks

1.1 Interrupt based Distance Measurements

Implement the distance measurement with the ultrasonic sensor interrupt based and without any polling or busy-waiting. Refer to the attached tutorial for the setup of your interrupt service routines.

- Step 1:** Use the interrupt service routine from a general purpose timer, configured in periodic mode, to trigger the measurement.
- Step 2:** Analyze your trigger signal with the oscilloscope to verify its functionality.
- Step 3:** Use another general purpose timer to measure the timing of the echo signal. Save the measured value in a global variable (has to be declared as *volatile*) and print it to the console.

Hint: Do not use any function calls in your interrupt service routines, as they take a significant amount of time. *Printf(..)* will most likely not work correctly inside of an ISR. Save your values in *volatile* global variables instead and print them in the main function.

1.2 Interrupt based Pendulum Display

Rewrite the code to display the measurements on the LED pendulum, so that it is interrupt based and works without any busy-waiting and polling.

Appendix 1: How do I configure my own Interrupt-Service-Routine?

In this lab, you will configure your own ISR Handler for GPIO and / or Timer Interrupts. You can find a code example for a GPIO-Interrupt in moodle. To setup your ISR Handler (let's call it *yourISRHandler* for this example), do the following steps:

1. Configure the Interrupt Source (GPIO / Timer / ...) Registers accordingly and enable the desired Interrupt in the Interrupt Mask Register (IM_R) of the GPIO Port / Timer / ...
2. Enable the desired Interrupt in the NVIC Enable Register (see page 116f, 154f of the datasheet)
3. Write your Interrupt Service Routine Function
4. Go to the file *tm4c1294ncpdt_startup_ccs.c* , search for the desired Interrupt and replace *IntDefaultHandler* with your own ISR-Function name (*yourISRHandler*).
5. If your ISR Handler function is contained in the *main.c* code file, you have to add the function declaration of your ISR Handler as „*extern void yourISRHandler(void);*“ to the top of the file *tm4c1294ncpdt_startup_ccs.c* . (See Interrupt Code example in Moodle)
6. If your ISR Handler function is contained within a header-file, you have to include this header-file in *tm4c1294ncpdt_startup_ccs.c*